Frappe / ERPNext

Docker Deployment Guide

A complete guide to installing, customizing,
and updating Frappe apps in production without a Docker registry

For Junior Developers

1. Core Concepts

Before touching any commands, understand these four ideas. Everything else builds on them.

Term	What it means
Image	A recipe baked once. Read-only. Contains your app code.
Container	A running instance of an image. Like a cake baked from a recipe.
Volume	Where your DATA lives. Survives restarts and image updates.
Docker Compose	A tool to run multiple containers together as one application.

The Golden Rule
Never edit code inside a running container — it disappears on restart.
Always bake changes into a new image, then redeploy.

The mental model for your workflow:

```
Your custom app code (pushed to GitHub)
        ↓
docker build  ←  reads apps.json, clones apps fresh from GitHub
```

```
        ↓
New image stored in Docker's internal library
        ↓
docker compose up  ←  picks up new image, recreates containers
        ↓
bench migrate  ←  applies any database changes
        ↓
Your data in volumes is untouched throughout
```

2. Folder Structure

You need two separate folders. They have completely different jobs.

```
~/
├── frappe_docker/              ← RUNNING: compose files live here
│   ├── compose.yaml
│   ├── overrides/
│   │   ├── compose.mariadb.yaml
│   │   ├── compose.redis.yaml
│   │   └── compose.proxy.yaml
│   ├── .env                   ← ONE .env file for everything
│   └── images/
│       └── layered/
│           ├── Containerfile  ← BUILDING: the recipe for your image
│           └── apps.json      ← BUILDING: lists which apps to install
```

Why two folders?

images/layered/ → Used only when building a new image. Touch it when updating apps.
frappe_docker/ → Used for deploying and running. Touch it for infra changes.

3. The .env File

One .env file in the frappe_docker/ root controls everything — both building and deploying.

```
# .env
```

```
# ── Image ──────────────────────────────────────
CUSTOM_IMAGE=frappe-custom
CUSTOM_TAG=v1.0.0
PULL_POLICY=never          # use local image, don't look online

# ── Database ───────────────────────────────────
DB_PASSWORD=yourpassword

# ── Site ───────────────────────────────────────
SITES=your-site.com

# ── GitHub token for private repos ─────────────
GITHUB_TOKEN=your_token_here
```

> ⚠ **Never commit .env to git**
> Your .env contains passwords and tokens. Make sure .env is in your .gitignore file.
> Run: echo '.env' >> .gitignore

---

4. The apps.json File

This file tells the image builder which Frappe apps to install. Save it at images/layered/apps.json.

Use GITHUB_TOKEN as a placeholder for your real token. It gets injected at build time.

```
[
  {
    "url": "https://github.com/frappe/erpnext",
    "branch": "version-15"
  },
  {
    "url":
"https://YOUR_USERNAME:GITHUB_TOKEN@github.com/yourorg/custom_app.git",
    "branch": "main"
  },
  {
    "url":
"https://YOUR_USERNAME:GITHUB_TOKEN@github.com/yourorg/another_app.git",
    "branch": "main"
  }
```

```
]
```

Important notes about apps.json

JSON does not support comments. To disable an app, remove it from the list.
Keep a separate apps.full.json as your master list for reference.
Always verify the branch name matches your repo (main, master, version-15, etc).
Public repos do not need a token in the URL.

---

5. Fresh Installation (First Time Setup)

Step 1 — Build your custom image

Run these commands from inside the frappe_docker/ directory:

```
cd ~/frappe_docker

# Load .env variables into your shell
source .env

# Inject token into apps.json and encode to base64
export APPS_JSON_BASE64=$(sed "s/GITHUB_TOKEN/$GITHUB_TOKEN/g" \
  images/layered/apps.json | base64 -w 0)

# Build the image
docker build \
  --build-arg=APPS_JSON_BASE64=$APPS_JSON_BASE64 \
  --tag=frappe-custom:v1.0.0 \
  --file=images/layered/Containerfile .
```

This step pulls your app code fresh from GitHub and bakes it into the image. It may take several minutes.

Step 2 — Start the application

```
docker compose \
  -f compose.yaml \
  -f overrides/compose.mariadb.yaml \
```

```
  -f overrides/compose.redis.yaml \
  -f overrides/compose.proxy.yaml \
  up -d
```

Wait for all containers to show as healthy:

```
docker compose ps
```

Step 3 — Create your site

```
docker compose exec backend \
  bench new-site your-site.com \
  --db-root-password yourdbpassword \
  --admin-password youradminpassword
```

Step 4 — Install apps on the site

Installing the image bakes the code in. This step activates each app on your specific site:

```
docker compose exec backend bench --site your-site.com install-app erpnext
docker compose exec backend bench --site your-site.com install-app mortuary_app
docker compose exec backend bench --site your-site.com install-app
custom_payments
# ... repeat for each custom app
```

Building vs Installing

docker build → bakes the app code into the image (runs once per version)
install-app → activates the app on a site and runs its database setup (runs once per site)

6. Updating an Existing App's Code

When you have pushed new code to GitHub for an app that is already installed.

Step 1 — Push your latest code to GitHub

On your development machine:

```
git add .
git commit -m "describe your changes"
git push
```

Step 2 — Rebuild the image with a new version tag

```
cd ~/frappe_docker
source .env

export APPS_JSON_BASE64=$(sed "s/GITHUB_TOKEN/$GITHUB_TOKEN/g" \
  images/layered/apps.json | base64 -w 0)

# Note: bump the tag from v1.0.0 to v1.0.1
docker build \
  --build-arg=APPS_JSON_BASE64=$APPS_JSON_BASE64 \
  --tag=frappe-custom:v1.0.1 \
  --file=images/layered/Containerfile .
```

Step 3 — Update .env with the new tag

```
CUSTOM_TAG=v1.0.1
```

Step 4 — Redeploy

```
docker compose \
  -f compose.yaml \
  -f overrides/compose.mariadb.yaml \
  -f overrides/compose.redis.yaml \
  -f overrides/compose.proxy.yaml \
  up -d
```

Step 5 — Run migrations

If your app has any database schema changes:

```
docker compose exec backend bench --site your-site.com migrate
```

Your data is safe
The database and uploaded files live in Docker volumes.
Redeploying only replaces the app code. Your data is never touched.

7. Adding a Brand New Custom App

When you want to add an app that was not in your original build.

Step 1 — Add the app to apps.json

```
[
  { "url": "https://github.com/frappe/erpnext", "branch": "version-15" },
  { "url": "https://user:GITHUB_TOKEN@github.com/yourorg/existing_app.git",
"branch": "main" },

  // Add your new app here:
  { "url": "https://user:GITHUB_TOKEN@github.com/yourorg/new_app.git", "branch":
"main" }
]
```

Step 2 — Rebuild, redeploy, then install the new app on the site

```
# Rebuild with new tag
source .env
export APPS_JSON_BASE64=$(sed "s/GITHUB_TOKEN/$GITHUB_TOKEN/g" \
  images/layered/apps.json | base64 -w 0)

docker build \
  --build-arg=APPS_JSON_BASE64=$APPS_JSON_BASE64 \
  --tag=frappe-custom:v1.0.2 \
  --file=images/layered/Containerfile .

# Update .env: CUSTOM_TAG=v1.0.2

# Redeploy
docker compose -f compose.yaml -f overrides/compose.mariadb.yaml \
  -f overrides/compose.redis.yaml -f overrides/compose.proxy.yaml up -d
```

```
# Install the new app on your site
docker compose exec backend bench --site your-site.com install-app new_app

# Run migrations
docker compose exec backend bench --site your-site.com migrate
```

8. Rolling Back to a Previous Version

Because you tag every build, rolling back is just changing one line in .env:

```
# In .env, change back to the previous tag
CUSTOM_TAG=v1.0.0   # was v1.0.1

# Redeploy
docker compose \
  -f compose.yaml \
  -f overrides/compose.mariadb.yaml \
  -f overrides/compose.redis.yaml \
  -f overrides/compose.proxy.yaml \
  up -d
```

The old image is still in Docker's internal storage. Nothing was deleted.

9. Quick Reference Commands

Command	What it does
docker compose ps	See status of all running containers
docker compose logs backend	See logs for the backend container
docker compose logs configurator	Debug startup issues
docker compose exec backend bash	Open a shell inside the backend container
docker compose down	Stop all containers (data is safe)
docker compose down -v	Stop and DELETE all data (careful!)
docker images	See all images stored in Docker's library

docker rmi frappe-custom:v1.0.0	Delete an old image to free up space

10. Common Mistakes to Avoid

✕Editing code inside a running container
Problem: changes disappear on the next restart.
Fix: push to GitHub, rebuild the image, redeploy.

✕Forgetting to bump the image tag
Problem: Docker Compose sees the same tag and does not recreate containers.
Fix: always increment the version tag with every build (v1.0.0 → v1.0.1 → v1.0.2).

✕Not running bench migrate after an update
Problem: app code is updated but the database schema is out of date, causing errors.
Fix: always run bench --site your-site.com migrate after redeploying.

✕Leaving PULL_POLICY unset when using a local image
Problem: Docker Compose tries to pull the image from Docker Hub, fails, and errors out.
Fix: always set PULL_POLICY=never in .env when using locally built images.

✕Committing .env to git
Problem: your database password and GitHub token are exposed publicly.
Fix: add .env to .gitignore immediately.